DESIGN DOCUMENT

Payment Orchestration System

Table of Contents

DESIGN DOCUMENT ***1***

INTRODUCTION **3**

Problem Statement **3**

System Overview **4**

System Guarantees **4**

Architecture Diagram **5**

End-to-End Flow **5**

Internal Processing And Component Behavior **11**

Request Ingestion (Synchronous Layer) 11
Execution Layer (Asynchronous Refund Processing) 13
External Confirmation Layer 13
Combined Guarantees 14
Ledger (Financial Source of Truth) 14
Projection Layer (Derived State Computation) 15

Idempotency & Deduplication **15**

Request Idempotency (Payment Creation) 15
Request Idempotency (Refund Creation) 16
Webhook Ingestion Idempotency 16
Webhook Processing Idempotency 17
Reconciliation Idempotency 17
Ledger-Level Deduplication 17
Projector Idempotency 18

Concurrency & Consistency **18**

Concurrency Model 19
Strong Consistency for Refund Initiation 19
Reservation-Based Validation 19
Although the ledger is the source of truth, refund validation relies on the derived state (payment_intent) for performance. 20
- Row-level locking ensures sequential validation 20
Handling Stale Derived State 20
Transactional Guarantees 20
Concurrency in Background Processing 21
Safe Event Convergence 21

Eventual Consistency Model 21
Deterministic State Reconstruction 22

Failure Handling & Recovery 22
Payment Intent Creation 22
Refund Creation 23
Refund Execution (Worker) 25
Webhook Ingestion 26
Webhook Processing 27
Reconciliation 27
Ledger Write Failure 28
Projector Failure 28

Data Model 29
payment_intent (Derived State) 29
refund_record (Intent + Reservation Layer) 30
ledger_entries (Financial Source of Truth) 30
webhook_events (Event Ingestion Layer) 31
projector_offsets 32
Relationships 32
Data Flow Overview 32
Design Principles 33

Design Trade-offs 33
Ledger-based Architecture vs Direct State Mutation 33
Eventual Consistency vs Strong Consistency (Read Model) 34
Row-Level Locking vs Optimistic Concurrency 34
Reservation Model (refund_record) vs Direct Refund Execution 35
At-least-once Processing vs Exactly-once Processing 35
Database as Coordination Layer vs External Queue (Kafka) 36
Strong Consistency for Writes vs Eventual Consistency for Reads 36
Idempotency Enforcement at Multiple Layers 37
Replayability vs Storage Overhead 37
Simplicity vs Extensibility 38

Scaling Considerations 38
Database Scaling 39
Worker Scaling 39
Projector Scaling 39
Ledger Growth 40
External PSP Scaling 40
Webhook & Event Processing 41
Hotspot Handling 41
Multi-Region Scaling (Future) 42
Summary 42

Limitations 42
Database Bottleneck 42
Worker Polling Inefficiency 43
Projector Throughput Limitation 43
Eventual Consistency of Read Model 43
Hotspot Contention per Payment 43
Ledger Growth 43
Tight Coupling to PostgreSQL 44
Single Region Deployment 44

INTRODUCTION

This document describes the design of the Payment Orchestration System.

The system is built to process payments and refunds in a reliable and consistent manner. It focuses on ensuring that financial data remains correct even when operations are retried, partially executed, or interrupted by failures.

The document explains how the system is structured, how data flows through it, and how different components work together to maintain correctness.

It covers the core parts of the system such as the ledger, the projector, and the derived payment state. It also describes the reasoning behind the design choices made to ensure safe and deterministic behavior.

Details unrelated to the core system design, such as user interfaces, infrastructure setup, or external integrations, are intentionally not included.

Problem Statement

This system is designed for a merchant-side payment flow where a client initiates a payment and confirms it through an external payment provider.

In such systems, operations are subject to retries, failures, and concurrent requests.

A single request may be executed multiple times due to network issues or timeouts. Without proper handling, this can lead to duplicate financial effects such as multiple captures or refunds.

Failures can occur during processing. If a system crashes after partially updating state, it may leave the payment in an inconsistent condition.

In addition, multiple operations can act on the same payment at the same time. The system must ensure that such concurrent actions do not violate financial constraints.

The system must guarantee that:

- Repeated execution does not create duplicate effects
- Partial failures do not corrupt state
- Concurrent operations do not violate financial limits
- The current state can always be derived deterministically from past actions

The problem is to design a system that maintains correctness under these conditions.

System Overview

The system follows an event-driven architecture where all financial actions are recorded as events, and the current state is derived from those events.

Instead of directly updating the payment state during request handling, each operation such as a payment or refund is first written to an append-only ledger. This ledger acts as the single source of truth for the system.

A background component, referred to as the projector, continuously reads events from the ledger and applies them to compute the current state of each payment. The computed state is stored in a separate table that represents the latest status of the payment.

Reads for user interfaces and dashboards are served from this derived state, which provides a fast and query-friendly view of the system. Financial correctness is based on the ledger, ensuring that decisions rely on immutable and verifiable data.

At a high level, the system operates as follows:

- Incoming requests generate events (PAYMENT / REFUND) that are stored in the ledger
- The projector reads these events in sequence
- The projector updates the derived payment state
- Reads are served from the derived state

This separation between event storage and state computation ensures that all changes are traceable and replayable, and that the system remains consistent under retries and failures.

System Guarantees

The system provides the following guarantees:

- **No duplicate financial effects**
  Repeated requests or events do not result in multiple captures or refunds.

- **No partial state visibility**
  All critical operations are atomic and either fully applied or fully rolled back.

- **Deterministic recovery**
  System state can always be reconstructed from the ledger.

- **No over-refund**
  Concurrent refund requests cannot exceed captured amount.

- **Eventual consistency for reads**
  Read models may lag but will converge to the correct state.

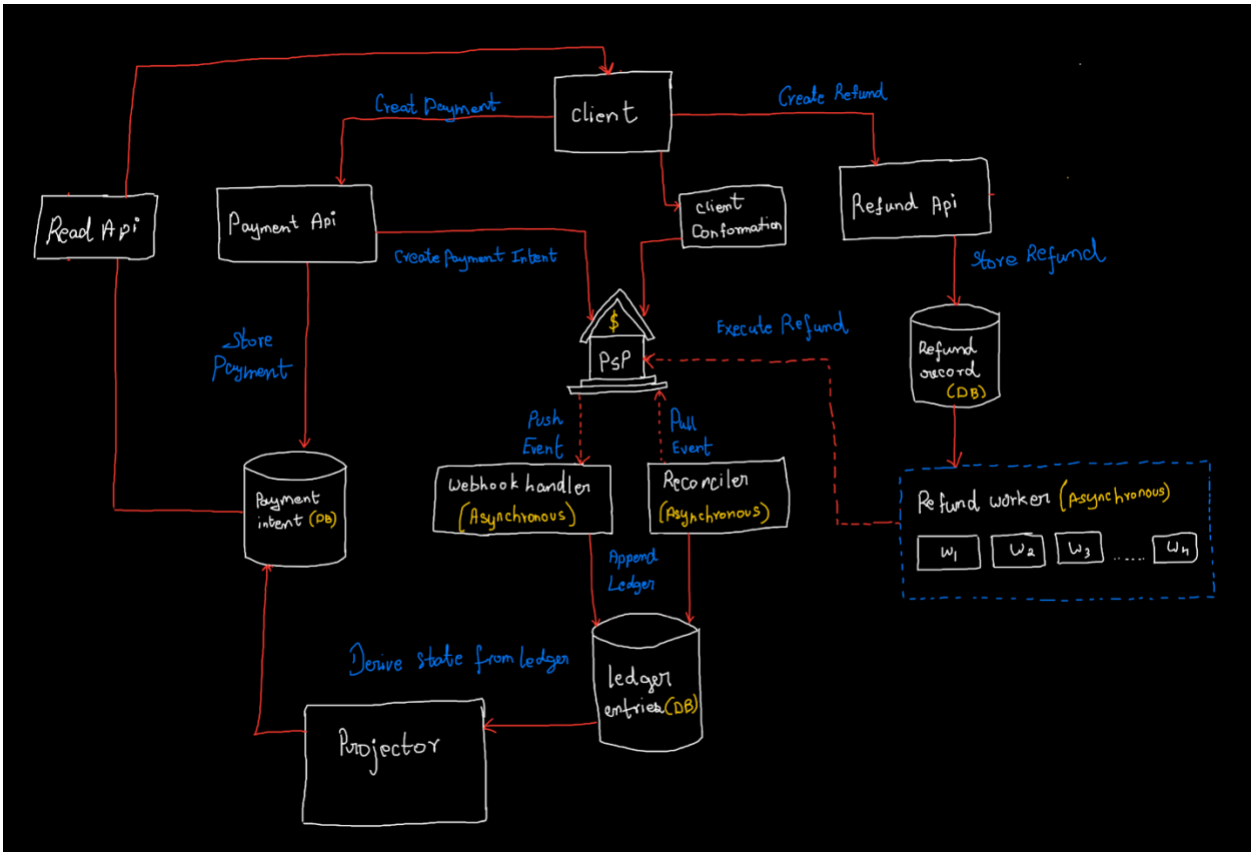
- **At-least-once processing with idempotency**
  Duplicate processing is allowed but does not affect correctness.

Architecture Diagram

This diagram shows the high-level architecture of the Payment Orchestration System.

The system follows an event-driven design where all financial actions are recorded in a ledger, and the current payment state is derived asynchronously.

External payment providers are integrated using asynchronous confirmation via webhooks and reconciliation.

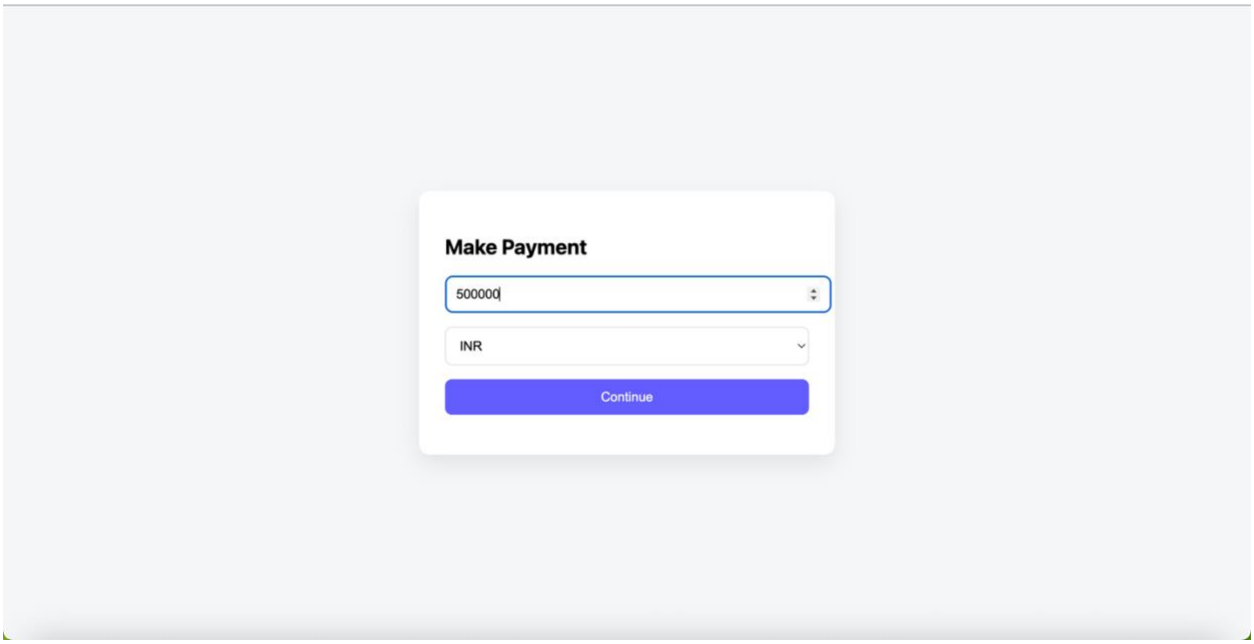


End-to-End Flow

This section describes how a payment flows through the system from user interaction to final state, including asynchronous confirmation and eventual consistency.

Step 1: Payment Initiation

The user enters the payment amount and currency and initiates the payment.



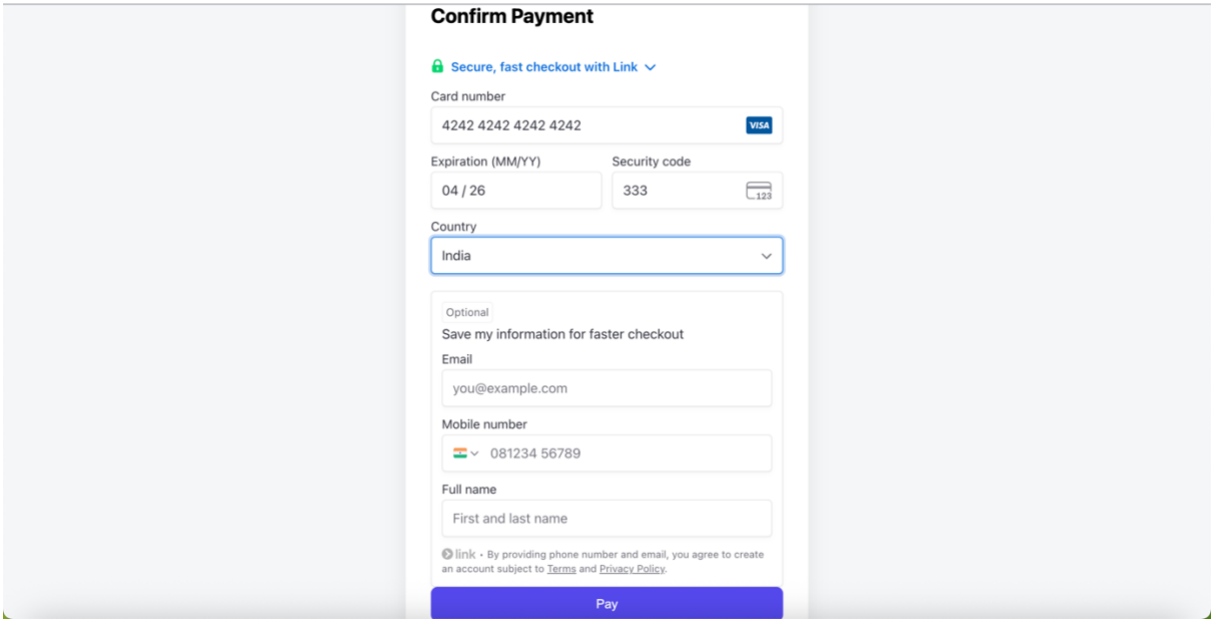
At this stage:

- A payment_intent record is created
- captured_amount = 0
- refunded_amount = 0

No financial action has occurred yet.

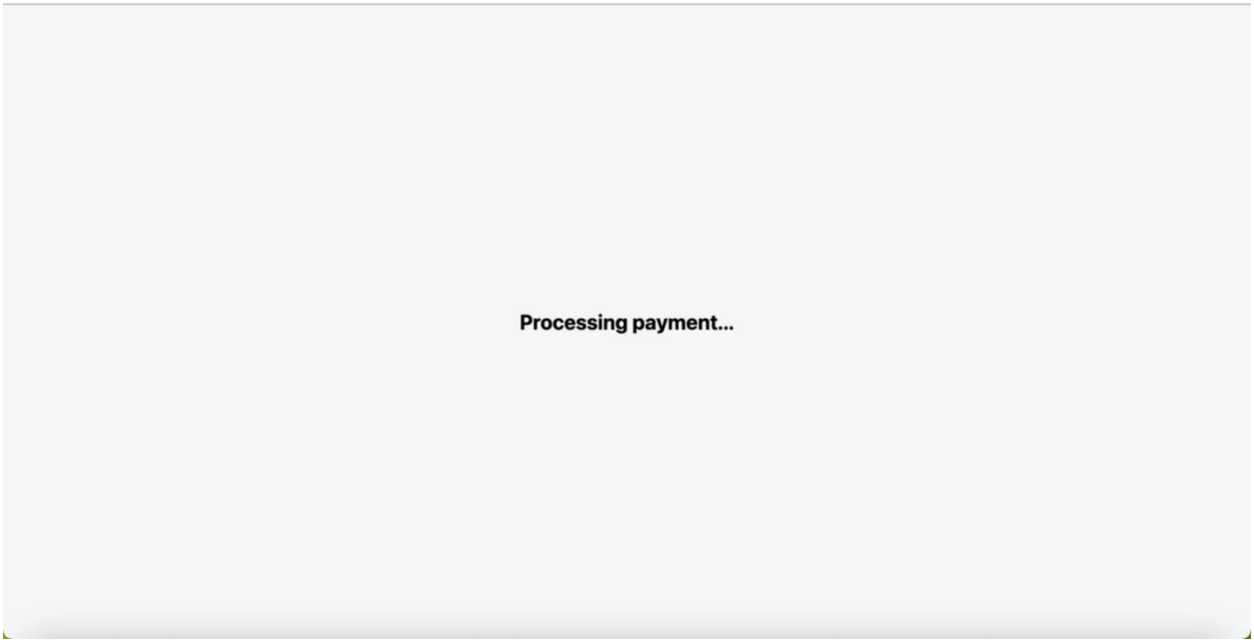
Step 2: Payment Processing via External Provider

The user is redirected to the external payment provider to complete the transaction.



After submitting payment details, the system waits for confirmation from the external provider.

If confirmation is not immediately available, the user is shown a processing screen.



This indicates:

- The payment has been initiated
- The system is awaiting confirmation from the external provider
- No PAYMENT event has been written to the ledger yet

Step 3: Asynchronous Payment Confirmation

The final payment outcome is received asynchronously through one of the following mechanisms:

- Webhook (push-based): The external provider sends a callback with the result
- Reconciliation (pull-based): The system periodically queries the provider for pending payments

Once confirmation is received:

- A PAYMENT event is written to the ledger
- This represents the confirmed financial action

Step 4: Projector Processing (Payment)

The projector reads the PAYMENT event from the ledger and updates the derived state.

- captured_amount is increased
- status is updated (e.g., CAPTURED)

This operation is idempotent. The projector tracks applied events to ensure that duplicate events are safely ignored.

The updated state is written to the derived table, which is used to serve read requests

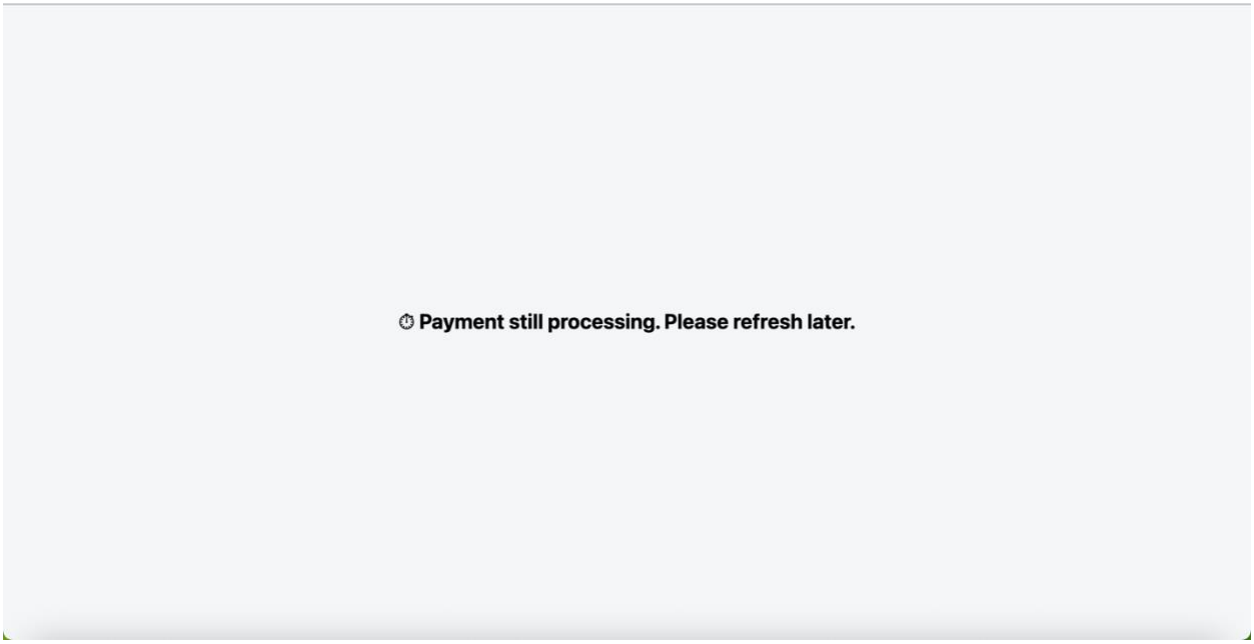
Step 5: Client Polling and State Visibility

After redirection from the payment provider, the client polls the backend to retrieve the latest payment status.

Polling is performed using an exponential backoff strategy to reduce load.

- If the PAYMENT event has been processed, the updated state is returned
- If not, the system continues to return a PROCESSING state

If confirmation is delayed, the user is informed that the payment is still processing and can check back later.



Step 6: Refund Initiation

A refund can be initiated from multiple sources:

- User-initiated: The user requests a refund
- System/merchant-initiated: The system triggers a refund due to business conditions such as:

    a) Order cancellation (e.g., out of stock)
    b) Duplicate payment detection
    c) Reconciliation or operational issues

All refund requests follow the same processing pipeline regardless of the source.

The system receives the refund request along with:

- payment_id
- refund amount
- idempotency key

Step 7: Validation and Refund Intent Creation

The system processes the request within a database transaction and locks the corresponding payment record to ensure concurrency safety.

Validation is performed using:

- The ledger (source of truth for completed financial actions)
- The refund_record table (to track in-flight refunds)

The system computes:

remaining_refundable = captured_amount − refunded_amount − pending_refunds

Where:

- captured_amount and refunded_amount are derived from the ledger
- pending_refunds are derived from refund_record entries in PENDING or PROCESSING state

If the requested amount exceeds the remaining refundable amount, the request is rejected.

If valid, a refund intent is created in the refund_record table with status PENDING.

This operation is idempotent:

- New request → new refund_id
- Retry → existing refund_id returned
- Conflict → rejected

No financial action has occurred at this stage. The refund intent represents a reserved operation.

Step 8: Refund Execution (Worker Processing)

A background worker processes refund intents using a leasing pattern.

The worker:

- Claims the refund intent
- Calls the external payment provider
- Stores the external reference (psp_refund_id)

The refund status transitions:

PENDING → PROCESSING

Step 9: Asynchronous Refund Confirmation

The final refund outcome is received asynchronously through:

- Webhook (push-based)
- Reconciliation (pull-based)

Once the refund is successfully confirmed:

- A REFUND event is appended to the ledger

This represents the confirmed financial outcome.

Step 10: Projector Processing (Refund)

The projector reads the REFUND event and updates the derived payment state.

- refunded_amount is increased
- status is updated (PARTIALLY_REFUNDED or REFUNDED)

This operation is idempotent. The projector tracks applied events to ensure that duplicate events are safely ignored.

Step 11: State Read

The updated payment state is served from the derived table.

This state is used by dashboards, APIs, and user interfaces.

At any point, the system can reconstruct the current state by replaying ledger events.

Internal Processing And Component Behavior

This section describes how the system processes requests internally across its components. The system operates as a pipeline, where each stage has a clear responsibility and ensures correctness under retries, failures, and concurrent operations.

Request Ingestion (Synchronous Layer)

1. **Payment Initiation**

   All operations enter the system through the API layer.

   For payment initiation, the system validates the request and interacts with the external payment provider to create a payment intent. The provider returns a reference and client secret, which is returned to the client to complete the payment.

A payment_intent record is created with initial values. This represents an intent to perform a payment, but no financial action has been recorded yet.

The system ensures that repeated requests do not create duplicate operations by handling requests in an idempotent manner.

2 Refund Initiation

Refund initiation is handled synchronously but does not immediately trigger a financial operation.

When a refund request is received, the system processes it within a database transaction to ensure consistency under concurrent requests. The corresponding payment record is accessed in a controlled manner to prevent race conditions.

Validation is performed using two sources:

- The ledger, which represents confirmed financial activity
- The refund_record table, which tracks in-flight refund operations

The system computes the remaining refundable amount as:

- remaining_refundable = captured_amount − refunded_amount − pending_refunds

Where:

- captured_amount is derived from PAYMENT events in the ledger
- refunded_amount is derived from REFUND events in the ledger
- pending_refunds are computed from refund_record entries in PENDING or PROCESSING state

The refund request is accepted only if:

- requested_amount ≤ remaining_refundable

This ensures that:

- The total refunded amount never exceeds the captured amount
- Concurrent refund requests do not violate financial limits
- In-flight refunds are accounted for during validation

If valid, a refund_record entry is created with a PENDING status. This acts as a reservation of funds and ensures safe future execution.

The operation is idempotent. Repeated requests with the same idempotency key return the same refund record.

No external financial action is performed at this stage.

Execution Layer (Asynchronous Refund Processing)

Refund execution is handled asynchronously by a background worker.

The worker continuously scans for refund_record entries in a PENDING state and claims them using a leasing mechanism to prevent duplicate execution. Once claimed, the worker invokes the external payment provider to execute the refund.

After initiating the refund, the system stores the external reference (psp_refund_id) in the refund_record.

The system does not assume success at this point and waits for confirmation from the external provider.

This separation between intent and execution ensures that retries and failures do not lead to duplicate financial effects.

External Confirmation Layer

The system relies on asynchronous confirmation from the external payment provider for both payments and refunds. This is handled through webhook processing and reconciliation.

1. **Webhook Processing**

   Webhook handling is the primary mechanism for receiving updates from the external payment provider.

   When a webhook event is received, the system first attempts to persist the event in a durable store.

   - If the event is successfully persisted, the system responds with HTTP 200, indicating that the event has been accepted and should not be retried.
   - If persistence fails, the system responds with an error, allowing the provider to retry delivery.

   This ensures that events are only acknowledged after being safely recorded.

Persisted webhook events are processed asynchronously by a background worker. The worker reads stored events, validates them, and appends the corresponding financial action to the ledger.

This design ensures reliable ingestion and safe processing under retries

2. **Reconciliation**

Reconciliation acts as a fallback mechanism to ensure no financial events are missed.

The system periodically scans for unresolved operations by inspecting:

- payment_intent records (for payments awaiting confirmation)
- refund_record entries (for refunds awaiting confirmation)

For such records, a background worker queries the external payment provider to fetch the latest status.

If a completed operation is identified, the system appends the corresponding event to the ledger.

This ensures recovery from missed webhooks and external inconsistencies.

Combined Guarantees

Webhook processing and reconciliation together ensure that:

- No financial event is lost
- Duplicate events do not create duplicate effects
- The system eventually reaches a correct and consistent state

Ledger (Financial Source of Truth)

All confirmed financial actions are recorded in the ledger as immutable events.

Each entry represents a completed financial operation, such as a payment or refund. The ledger is append-only and does not allow modification of past events.

This ensures:

- A complete and auditable financial history
- Recovery from failures through replay
- Strong guarantees of financial correctness

The ledger is the single source of truth for the system.

Projection Layer (Derived State Computation)

The current state of each payment is derived from the ledger using a background component called the projector.

The projector continuously reads events from the ledger and applies them in order to compute the latest state, including captured amounts, refunded amounts, and status.

Processing is safe under retries, ensuring that repeated application of the same event does not alter the final result.

The derived state is stored in a separate table optimized for reads, which is used by APIs, dashboards, and client polling.

Idempotency & Deduplication

The system is designed to remain correct under retries, duplicate requests, and repeated external events. Since both clients and external payment providers operate in unreliable environments, every layer of the system enforces idempotent behavior to ensure that financial effects occur exactly once.

Idempotency is not handled at a single point, but is enforced consistently across request ingestion, event ingestion, processing, and state derivation.

Request Idempotency (Payment Creation)

Payment creation is made idempotent by associating each request with a deterministic fingerprint derived from its input parameters.

For each incoming request:

- A hash is computed using fields such as amount, currency, and PSP
- The system attempts to persist the request along with this hash
- If the same request is retried with identical parameters, the existing payment is returned
- If the same logical request is retried after partial execution, the system resumes using stored state

If a request is retried after the external payment intent has already been created:

- The system detects the existing PSP reference
- Instead of creating a new payment intent, it fetches the existing state from the provider

If the same idempotency key or request is reused with different parameters:

- The request is rejected to prevent conflicting operations

This ensures that:

- Duplicate client retries do not create multiple payment intents
- Partial failures do not result in inconsistent external state
- Each logical payment operation maps to a single external intent

Request Idempotency (Refund Creation)

Refund creation is enforced using a client-provided idempotency key.

The system guarantees uniqueness using the combination of (payment_id, idempotency_key):

- A refund_record is inserted with status PENDING
- If a record already exists for the same key, the existing record is returned
- If the same key is reused with different parameters, the request is rejected

This logic is executed within a database transaction, ensuring atomicity under concurrent requests.

Additionally:

- The refund is validated against both the ledger and in-flight refunds
- A reservation model is used to prevent over-refund

This ensures that:

- Retried refund requests do not create duplicate refund intents
- Concurrent refund attempts remain safe
- Each refund operation is executed exactly once from the client's perspective

Webhook Ingestion Idempotency

External payment providers deliver webhook events using an at-least-once delivery model, meaning the same event may be delivered multiple times.

To handle this:

- Each webhook event is uniquely identified using (event_id, psp_name)
- Events are persisted using an upsert mechanism (ON CONFLICT DO NOTHING)
- The system acknowledges the webhook (HTTP 200) only after successful persistence

If persistence fails:

- The system returns an error response
- The provider retries delivery

This ensures that:

- No event is acknowledged unless it is safely recorded
- Duplicate webhook deliveries do not result in duplicate stored events

Webhook Processing Idempotency

Persisted webhook events are processed asynchronously by background workers.

Processing guarantees:

- Events are claimed using a leasing mechanism to avoid concurrent processing
- Each event is processed within a database transaction
- Ledger updates and state transitions are applied atomically

If processing fails:

- The event remains available for retry
- After repeated failures, it may be marked as failed

Since downstream operations (ledger writes and updates) are idempotent:

- Retrying the same webhook event does not create duplicate financial effects

Reconciliation Idempotency

Reconciliation periodically queries the external payment provider for unresolved operations.

Since the same external state may be retrieved multiple times:

- Reconciliation is designed to be safely repeatable
- Before recording any financial event, the system ensures it has not already been recorded

This ensures that:

- Reconciliation can be retried indefinitely without side effects
- Missing webhook events are safely recovered
- Duplicate external reads do not result in duplicate ledger entries

Ledger-Level Deduplication

The ledger enforces uniqueness of financial events at the database level.

Each event is associated with a unique external reference (e.g., PSP payment or refund ID).

When inserting into the ledger:

- A uniqueness constraint ensures that the same external event cannot be recorded more than once
- Duplicate insert attempts are ignored

This guarantees that:

- Each real-world financial action is recorded exactly once
- Duplicate processing from webhook or reconciliation does not corrupt financial data

Projector Idempotency

The projector derives the current state from ledger events.

To ensure correctness under retries:

- Each payment tracks the last applied event sequence (last_applied_seq)
- Events are applied only if their sequence is greater than the last applied value

During processing:

- Duplicate or already-applied events are skipped
- State updates are applied deterministically

This ensures that:

- Reprocessing events does not change the final state
- Partial failures do not result in incorrect state
- The derived state always remains consistent with the ledger

Concurrency & Consistency

The system is designed to remain correct under concurrent execution, where multiple requests and background processes may operate on the same payment simultaneously.

Instead of restricting concurrency, the system allows parallel execution while enforcing correctness through transactional guarantees, row-level locking, and deterministic validation based on the ledger.

Concurrency Model

The system operates in a highly concurrent environment with multiple independent actors:

- Multiple client requests (e.g., concurrent refund/ payment attempts)
- Webhook workers processing external events
- Reconciliation workers querying external systems
- Projector updating derived state

These components operate independently and concurrently. The system ensures that their interactions do not violate financial correctness.

Strong Consistency for Refund Initiation

Refund initiation is a critical operation that must remain consistent under concurrent requests.

To ensure this:

- The system processes refund creation inside a database transaction
- The corresponding payment row is locked using SELECT ... FOR UPDATE

This guarantees that:

- Only one transaction can validate and reserve refundable funds at a time
- Concurrent refund requests cannot observe stale or inconsistent state

Reservation-Based Validation

Refund validation is performed using a reservation model.

The system computes:

remaining_refundable = captured_amount − refunded_amount − pending_refunds

Where:

- captured_amount and refunded_amount are derived from the projected state(Payment_intent)
- pending_refunds are computed from refund_record entries in PENDING or PROCESSING state

This ensures that:

- In-flight refunds are accounted for before confirmation

- Concurrent requests cannot over-allocate funds
- Financial limits are enforced deterministically

Note:

Although the ledger is the source of truth, refund validation relies on the derived state (payment_intent) for performance.

Correctness is preserved because:

- Row-level locking ensures sequential validation
- Pending refunds act as a reservation layer
- Validation subtracts both committed and in-flight amount

As a result, even if the derived state is slightly stale, over-refund is prevented

Handling Stale Derived State

The derived state (payment_intent) is updated asynchronously and may be slightly stale.

The system remains correct despite this because:

- Row-level locking ensures sequential validation
- The reservation layer (pending_refunds) tracks already allocated amounts
- Validation subtracts both committed and in-flight values

As a result:

- Even if refunded_amount is stale, over-refund is prevented
- The system remains safe under eventual consistency

Transactional Guarantees

All critical operations are executed within database transactions, including:

- Refund creation and validation
- Webhook event processing
- Ledger writes
- Reconciliation updates
- Projector state updates

This ensures that:

- Partial updates are never visible
- Failures result in complete rollback

- The system does not enter inconsistent intermediate states

Concurrency in Background Processing

Background workers operate concurrently to process events and reconciliation tasks.

To ensure safe execution:

- Work items are claimed using a leasing pattern (FOR UPDATE SKIP LOCKED)
- Each worker processes only the records it has claimed
- Multiple workers can operate in parallel without duplication

This guarantees that:

- No record is processed by multiple workers simultaneously
- Processing scales horizontally without coordination issues

Safe Event Convergence

The system receives updates from multiple asynchronous sources:

- Webhooks (push-based)
- Reconciliation (pull-based)

Both may attempt to process the same event concurrently.

To ensure correctness:

- Ledger enforces uniqueness constraints on external references
- Duplicate inserts are ignored

This guarantees that:

- Duplicate processing does not create duplicate financial effects
- The system converges to the correct state regardless of event ordering

Eventual Consistency Model

The system separates financial correctness from read visibility:

- Financial correctness is ensured by the ledger and transactional updates
- Read models (derived state) are updated asynchronously by the projector

As a result:

- Clients may temporarily observe stale state
- The system guarantees eventual consistency

This trade-off enables:

- High write reliability
- Scalable read performance

Deterministic State Reconstruction

All state transitions are derived from immutable ledger events.

This ensures that:

- The current state can always be reconstructed by replaying events
- Concurrent execution does not introduce ambiguity
- The system remains deterministic under retries and failures

Failure Handling & Recovery

The system is designed to remain correct under partial failures, retries, and unreliable external systems. Since payment processing involves multiple asynchronous components and external dependencies, failures are expected rather than exceptional.

Instead of attempting to prevent failures, the system is built to tolerate them and recover deterministically without introducing duplicate or inconsistent financial effects.

Payment Intent Creation

Flow

- User → API
- Persist payment_intent (CREATED) using request fingerprint
- Call PSP to create payment intent
- Persist psp_ref_id, move to PROCESSING
- Return client_secret

Scenario: Crash before PSP call

Recovery

- Client retries request
- Request is processed normally

Guarantee

- No external side effect

Scenario: Crash after PSP call, before persisting psp_ref_id

Recovery

- Retry → request hash match → existing record reused
- If psp_ref_id is missing → retry PSP CreatePaymentIntent
- PSP idempotency ensures same external intent

Guarantee

- No duplicate PSP intent
- External state remains consistent

Scenario: Crash after psp_ref_id is persisted

Recovery

- Retry detects existing psp_ref_id
- System calls PSP GetPaymentIntent
- Returns existing client_secret

Guarantee

- No duplicate PSP calls
- Deterministic response

Refund Creation

Flow

- User → API
- Begin transaction
- Lock payment_intent (SELECT ... FOR UPDATE)

- Insert refund_record (PENDING) using (payment_id, idempotency_key)
- Validate:
- remaining_refundable = captured_amount - refunded_amount - pending_refunds
- Update metadata
- Commit

Scenario: Crash before transaction begins

Recovery

- Retry starts a new transaction

Guarantee

- No side effect

Scenario: Crash during transaction (before commit)

Recovery

- Transaction rollback removes all changes
- Retry re-executes validation

Guarantee

- No partial reservation
- No incorrect refund allocation

Scenario: Concurrent refund requests

Recovery Mechanism

- Row-level lock serializes access
- Each request validates against latest state

Guarantee

- No over-refund
- Deterministic allocation

Scenario: Retry with same idempotency key

Recovery

- Insert conflict → existing record fetched

Guarantee

- Same refund_id returned
- No duplicate refund

Scenario: Retry with conflicting payload

Recovery

- Existing record fetched
- Payload mismatch detected

Guarantee

- Request rejected (idempotency_conflict)

Refund Execution (Worker)

Flow

- Claim refund (FOR UPDATE SKIP LOCKED)
- Call PSP
- Persist psp_refund_id, move to PROCESSING

Scenario: Worker crash after PSP call, before persistence

Recovery

- Retry triggers PSP call again
- PSP idempotency ensures same refund

Guarantee

- No duplicate refund

Scenario: Worker crash after persistence

Recovery

- Refund remains in PROCESSING
- Webhook or reconciliation completes flow

Guarantee

- Eventual completion

Webhook Ingestion

Flow

- Receive webhook
- Persist event
- Return HTTP 200

Scenario: Crash before persistence

Recovery

- PSP retries webhook

Guarantee

- No event loss

Scenario: Crash after persistence

Recovery

- Event processed asynchronously

Guarantee

- At-least-once processing

Webhook Processing

Flow

- Claim event
- Process inside transaction
- Write ledger / update state

Scenario: Processing failure

Recovery

- Event retried
- After max attempts → marked failed

Guarantee

- No partial updates

Reconciliation

Flow

- Scan unresolved records
- Query PSP
- Update ledger/state

Scenario: Missed webhook

Recovery

- Reconciliation detects and recovers

Guarantee

- No missed financial event

Ledger Write Failure

Scenario

- Insert into ledger fails

Recovery

- Transaction rollback
- Retry via webhook or reconciliation

Guarantee

- No partial financial state

Projector Failure

Scenario

- Projector crashes mid-processing

Recovery

- Resume from last processed sequence
- Replay ledger events

Guarantee

- No state corruption
- Eventually consistent state

Data Model

The system is designed around a clear separation between immutable financial events and derived state.

This separation ensures correctness, auditability, and deterministic recovery under failures.

The data model is structured into four logical layers:

- Intent & Request Layer
- Execution & Tracking Layer
- Ledger (Source of Truth)
- Projection (Derived State)

payment_intent (Derived State)

The payment_intent table represents the current state of a payment.

This state is derived asynchronously from ledger events by the projector.

Key Fields

- payment_id (PK)
- idempotency_key
- request_hash
- status (CREATED, PROCESSING, CAPTURED, FAILED, CANCELLED, PARTIALLY_REFUNDED, REFUNDED)
- amount, captured_amount, refunded_amount
- currency
- psp_name, psp_ref_id
- last_applied_seq
- created_at, updated_at

Purpose

- Serves as the read model for APIs and dashboards
- Stores the latest computed financial state
- Used for validation during refund initiation

Properties

- Mutable
- Eventually consistent
- Not the source of truth

refund_record (Intent + Reservation Layer)

The refund_record table represents refund intents and in-flight operations.

Key Fields

- refund_entry_id (PK)
- payment_id (FK)
- amount
- status (PENDING, PROCESSING, SUCCEEDED, FAILED)
- idempotency_key
- psp_refund_id, psp_payment_ref_id
- retry_count, max_retry, next_retry_at
- next_reconcile_at, reconcile_attempts
- created_at, updated_at

Constraints

- UNIQUE (payment_id, idempotency_key)

Purpose

- Tracks refund lifecycle from intent to completion
- Acts as a reservation layer to prevent over-refund
- Enables safe retries and worker-based execution

Properties

- Mutable
- Represents both intent and execution state

ledger_entries (Financial Source of Truth)

The ledger_entries table is the immutable financial ledger of the system.

Each row represents a confirmed financial event.

Key Fields

- seq (monotonic, PK)
- ledger_entry_id (UUID)
- payment_id (FK)
- refund_id (nullable FK)
- entry_type (PAYMENT, REFUND)
- amount, currency

- psp_name, psp_ref_id
- created_at

Constraints

- UNIQUE (psp_name, psp_ref_id, entry_type)
- Append-only enforced via triggers (no UPDATE / DELETE)

Purpose

- Stores all confirmed financial actions
- Acts as the single source of truth
- Enables replay, auditing, and recovery

Properties

- Immutable
- Strong consistency
- Append-only

webhook_events (Event Ingestion Layer)

The webhook_events table stores raw events received from external providers.

Key Fields

- id (PK)
- event_id, psp_name
- payload (JSONB)
- status (PENDING, PROCESSING, PROCESSED, FAILED)
- attempts, next_retry_at
- created_at, updated_at

Constraints

- UNIQUE (event_id, psp_name)

Purpose

- Ensures durable ingestion of external events
- Enables retry-safe asynchronous processing
- Decouples ingestion from business logic

projector_offsets

Tracks the progress of the projector.

Key Fields

- projector_name (PK)
- last_processed_seq
- updated_at

Purpose

- Enables exactly-once projection semantics
- Allows safe recovery after crashes

—

Relationships

- payment_intent.payment_id is the primary entity
- refund_record.payment_id references payment_intent
- ledger_entries.payment_id references payment_intent
- ledger_entries.refund_id references refund_record

Data Flow Overview

1. Intent Layer
    - payment_intent (initial creation)
    - refund_record (reservation)
2. Execution Layer
    - webhook_events
    - reconciliation
3. Source of Truth
    - ledger_entries (immutable events)
4. Projection Layer

    - payment_intent (derived state via projector)

Design Principles

Separation of Concerns

- Ledger = financial truth
- payment_intent = derived read model
- refund_record = intent + reservation

Immutability

- Financial events are never updated or deleted

Idempotency Enforcement

- request_hash (payment requests)
- (payment_id, idempotency_key) (refunds)
- (psp_name, psp_ref_id, entry_type) (ledger)

Deterministic Recovery

- State can be reconstructed by replaying ledger events

Concurrency Safety

- Row-level locking on payment_intent
- Reservation tracking via refund_record

Design Trade-offs

The system is built with a strong focus on financial correctness, determinism, and failure safety.

To achieve these guarantees, several deliberate trade-offs have been made across consistency, performance, and system complexity.

Ledger-based Architecture vs Direct State Mutation

Decision

Use an append-only ledger as the source of truth instead of directly mutating payment state.

Why chosen

- Ensures immutable financial history

- Enables replay and deterministic recovery
- Prevents corruption from partial failures

Trade-off

- Increased system complexity
- Requires asynchronous projection layer
- Higher write amplification

Why acceptable

Financial systems require auditability and correctness over simplicity. The ledger guarantees that no financial event is lost or overwritten.

Eventual Consistency vs Strong Consistency (Read Model)

Decision

Use asynchronous projection (projector) to derive payment state.

Why chosen

- Decouples write path from read path
- Enables scalable read performance
- Simplifies event replay and recovery

Trade-off

- Clients may observe stale state temporarily

Why acceptable

- Financial correctness depends on the ledger, not the read model
- Critical validations (e.g., refunds) account for in-flight operations
- Eventual consistency is sufficient for UI and polling flows

Row-Level Locking vs Optimistic Concurrency

Decision

Use SELECT ... FOR UPDATE to serialize refund validation.

Why chosen

- Prevents race conditions during refund allocation
- Ensures deterministic validation

Trade-off

- Reduced concurrency for the same payment
- Potential lock contention under high load

Why acceptable

- Contention is scoped per payment
- Financial correctness (no over-refund) is more critical than throughput
- Typical payment operations are low-frequency per entity

Reservation Model (refund_record) vs Direct Refund Execution

Decision

Introduce a reservation layer (refund_record) before executing refunds.

Why chosen

- Tracks in-flight refunds
- Prevents over-refund under concurrency
- Enables safe retries and worker-based execution

Trade-off

- Additional table and lifecycle management
- Increased system complexity

Why acceptable

Without reservations, concurrent refund requests could exceed captured amount. The reservation model guarantees safe allocation before execution.

At-least-once Processing vs Exactly-once Processing

Decision

Adopt at-least-once delivery for webhook and worker processing.

Why chosen

- Aligns with external PSP guarantees
- Simpler and more robust than enforcing exactly-once

Trade-off

- Duplicate events may be processed

Why acceptable

- Idempotency is enforced at every layer (DB constraints, ledger uniqueness)
- Duplicate processing does not result in duplicate financial effects

Database as Coordination Layer vs External Queue (Kafka)

Decision

Use the database as a coordination mechanism (row-level locking + FOR UPDATE SKIP LOCKED) for worker processing instead of introducing an external queue like Kafka.

Why chosen

- Simpler architecture with fewer moving parts
- Strong transactional guarantees (claim + update in same transaction)
- Easier reasoning about correctness and failure recovery

Trade-off

- Increased database load due to worker polling
- Limited horizontal scalability compared to distributed queues
- Tight coupling between processing and database performance

Why acceptable

- Contention is limited and scoped (e.g., refund-level or event-level)
- Financial correctness benefits from transactional coupling
- Current scale does not justify the complexity of Kafka
- Design allows future migration to a queue-based system if needed

Strong Consistency for Writes vs Eventual Consistency for Reads

Decision

Use strong consistency for all financial writes, and eventual consistency for read models.

Why chosen

- Financial operations (payments, refunds) must always be correct
- All writes are done inside transactions with proper locking
- Read model (payment_intent) is derived asynchronously for performance

Trade-off

- Read data (status, refunded_amount) may be slightly stale
- No strict read-after-write guarantee

Why acceptable

- Financial correctness is enforced during writes, not reads
- Refund validation accounts for in-flight operations (pending_refunds)
- Row-level locking prevents over-refund even with stale data
- Users can tolerate small delays in UI updates

Idempotency Enforcement at Multiple Layers

Decision

Enforce idempotency across:

- API layer (request hash / idempotency key)
- Database layer (unique constraints)
- Ledger (external reference uniqueness)

Why chosen

- Handles retries from clients and external systems
- Prevents duplicate financial effects

Trade-off

- Additional storage and validation logic
- Increased complexity in request handling

Why acceptable

Distributed systems cannot avoid retries. Multi-layer idempotency ensures safety under all failure modes.

Replayability vs Storage Overhead

Decision

Retain all ledger events for replay and auditing.

Why chosen

- Enables deterministic recovery

- Supports debugging and audit requirements

Trade-off

- Increased storage usage over time

Why acceptable

- Storage is cheaper than data loss
- Can be optimized with archival strategies

Simplicity vs Extensibility

Decision

Keep the system minimal (DB + workers) without introducing complex infrastructure (Kafka, distributed logs).

Why chosen

- Faster development and iteration
- Easier reasoning about correctness

Trade-off

- Limited scalability compared to distributed systems

Why acceptable

- System is designed to evolve
- Core correctness guarantees are already established

Scaling Considerations

The system is designed with correctness as the primary goal, but it allows incremental scaling as load increases. Scaling is achieved by isolating bottlenecks and evolving components independently.

Database Scaling

Current Design

- Single PostgreSQL instance handles:
    - payment_intent
    - refund_record
    - ledger_entries
    - webhook_events

Scaling Approach

- Read replicas for read-heavy operations (GET payment status)
- Vertical scaling for write-heavy workloads
- Partitioning tables by payment_id or time (ledger_entries)

Future Evolution

- Shard by payment_id to distribute load across databases

Worker Scaling

Current Design

- Workers poll DB using FOR UPDATE SKIP LOCKED

Scaling Approach

- Increase number of workers horizontally
- Each worker safely processes independent jobs

Limitations

- DB contention under high concurrency
- Polling overhead

Future Evolution

- Move to queue-based system (Kafka / message broker) for high throughput

Projector Scaling

Current Design

- Single projector processes ledger sequentially

Scaling Approach

- Batch processing of events
- Optimize queries and indexing

Limitations

- Single-threaded processing becomes bottleneck

Future Evolution

- Partition projector by payment_id (multiple projector workers)
- Each worker processes independent partitions

Ledger Growth

Problem

- Ledger is append-only → grows indefinitely

Scaling Approach

- Index optimization for query performance
- Partition ledger by time or payment_id

Future Evolution

- Archive old events to cold storage
- Keep recent data hot for fast access

External PSP Scaling

Current Design

- Direct API calls to PSP

Scaling Considerations

- Rate limits from PSP
- Network latency

Mitigation

- Retry with backoff
- Idempotent external calls
- Circuit breaker (future improvement)

Webhook & Event Processing

Current Design

- DB-backed event storage + workers

Scaling Approach

- Increase worker count
- Batch event processing

Limitations

- DB load increases with event volume

Future Evolution

- Move ingestion to message queue (Kafka)
- Decouple ingestion from processing

Hotspot Handling

Problem

- Single payment_id can become contention hotspot

Mitigation

- Row-level locking isolates contention
- Contention limited to single payment

Future Evolution

- Rate limiting per payment
- Queue per payment (logical partitioning)

Multi-Region Scaling (Future)

Current Design

- Single region deployment

Challenges

- Cross-region consistency
- Duplicate processing

Future Approach

- Region-based partitioning
- Global idempotency keys
- Eventually consistent cross-region reconciliation

Summary

The system scales by:

- Increasing workers for parallel processing
- Optimizing database through partitioning and replicas
- Moving to queue-based architecture when needed

The design intentionally starts simple and evolves toward distributed systems only when scale requires it, while preserving correctness guarantees.

Limitations

The current system is designed for correctness and simplicity, but has the following practical limitations.

Database Bottleneck

The database handles:

- Ledger storage
- Worker coordination
- Refund reservations

Limitation

- High load can cause contention and increased latency
- Throughput is limited by a single database instance

Worker Polling Inefficiency

Workers continuously poll the database for new jobs.

Limitation

- Wasted queries when no work is available
- Increased load on the database under scale

Projector Throughput Limitation

The projector processes ledger events sequentially.

Limitation

- Event processing lag increases under high write volume
- Derived state may fall behind the source of truth

Eventual Consistency of Read Model

The payment_intent table is asynchronously updated.

Limitation

- Users may see stale payment or refund status
- No immediate consistency after write operations

Hotspot Contention per Payment

Operations on the same payment_id are serialized using row-level locks.

Limitation

- High contention for frequently updated payments
- Limited parallelism for a single payment

Ledger Growth

The ledger is append-only and continuously grows.

Limitation

- Storage usage increases over time
- Query performance may degrade without maintenance

Tight Coupling to PostgreSQL

The system relies on:

- Transactions
- Row-level locking
- SKIP LOCKED

Limitation

- Reduced portability to other data stores
- System behavior tightly bound to database capabilities

Single Region Deployment

The system operates in a single region.

Limitation

- No fault tolerance across regions
- Regional outage impacts entire system